ETNA: EnTropy regularization for compressed Neural Avatar Supplementary Material

Guillaume Loranchet[1,2]

João Regateiro[1]

Bharath Bhushan Damodaran[1]

Franck Galpin[1]

Adnane Boukhayma[2]

Franck Multon[2]

Pierre Hellier[2]

[1] InterDigital
Rennes, France

[2] Univ Rennes, Inria, CNRS, IRISA
Rennes, France

This supplementary material provides additional implementation and evaluation details. Section 1 describes the soft-counting procedure used to compute the differentiable soft-entropy regularizer. Section 2 reports the training protocol and hyperparameters used for the main experiments. Section 3 summarizes the experimental setup, including the datasets, evaluation metrics, and the two bitstream coding tools used to compress the trained weights. Finally, Section 4 presents additional analyses and results, including experiments on the full DFaust dataset, the relationship between entropy and achieved file size, qualitative comparisons at low bitrates, and sensitivity to the entropy weight λ and post-training quantization.

1 Soft-counting details

Figure 1 summarizes the computation of our soft-counting method for a set of unquantized weights. In this work, all the learnable weights are concatenated into a single tensor, although for lighter networks, it is possible to learn per-layer distributions. After estimating the counts for all bins, the results are then normalized to obtain a probability distribution to compute the soft-entropy.

© 2026. The copyright of this document resides with its authors.
It may be distributed unchanged freely in print or electronic forms.

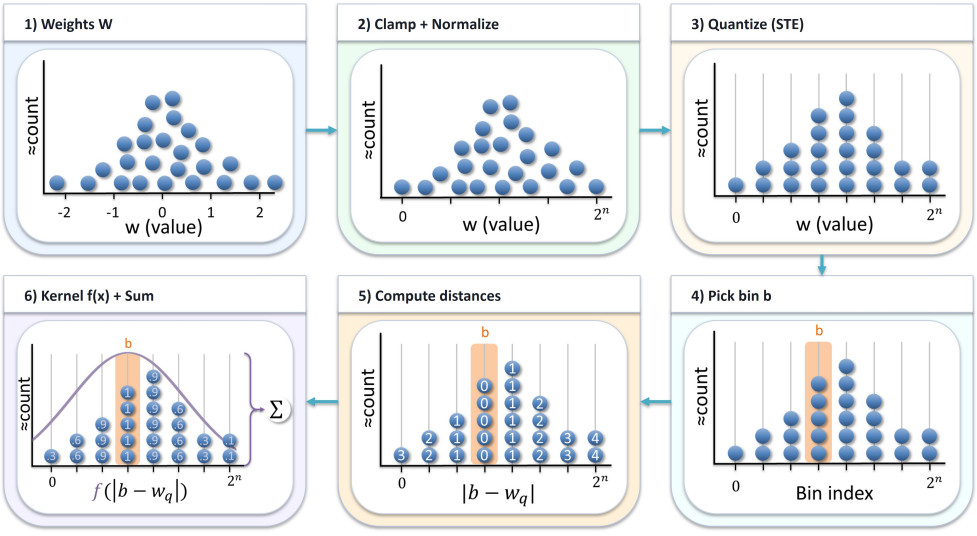


Figure 1: Overview of the soft-entropy computation. The unquantized weights (1) are clamped to fixed boundaries and normalized (2), then quantized using STE (3). For each bin b (4), the model computes the distance to all weights (5), applies the kernel function and sums the result to obtain a soft-count (6).

2 Implementation details

The training schedule depends on the underlying model. For brevity, we only present the procedure used to train LHB [8] but we follow the same strategy for COAP [9]. The model is initially trained on the reconstruction task only (without entropy regularization) for 150k steps. At 150k steps, the L1 loss and boundary penalty are then added. After an additional 5k steps, the entropy regularization is gradually introduced between 155k and 185k steps. Finally, the training is continued for 20k steps to stabilize the results.

For each training sample, the network predicts the occupancy of 3500 3D points. At the beginning of training, $7/8$ of the points are sampled close to the surface and $1/8$ are sampled uniformly in space. Similar to the procedure described in Fast-SNARF [5], the points close to the surface are sampled on the mesh surface before being perturbed by a Gaussian noise with a standard deviation of 0.01. The same number of points is sampled for each face, thus sampling more points in highly detailed regions. Over the course of training, the proportion of uniformly sampled points decreases to 1%, which improves the accuracy of predictions in challenging regions.

Optimization is performed with AdamW, using a weight decay of 0.01. The weight decay encourages the weight distribution to remain close to zero, making AdamW preferable to the classical Adam optimizer. The learning rate is scheduled with PyTorch's `CosineAnnealingLR`, and alternates between 10^{-3} and 2×10^{-5}. Stochastic Weight Averaging (SWA) is applied during the final phase to further stabilize training with a learning rate of 2×10^{-5}.

In all experiments, we set $\lambda_R = 100$, $\lambda_{L_1} = 10^4$, and $\lambda_{\max} = 100$. The parameter λ was used to control the trade-off between reconstruction quality and compression efficiency. By default, the weight boundaries are set to -2 and 2, and the kernel σ is set to 0.1. Table 1 summarizes the default hyperparameters used in the main experiments.

Parameter	Value
Quantization level during training	$q = 10$
Soft-count kernel	$\sigma = 0.1$
Weight boundaries	$[-2, 2]$
Reconstruction weight	$\lambda_R = 100$
L1 alignment weight	$\lambda_{L_1} = 10^4$
Boundary loss weight	$\lambda_{\max} = 100$
Entropy weight	varied to control bitrate
Optimizer	AdamW, weight decay 0.01
SWA learning rate	2×10^{-5}

Table 1: Default training parameters used in the main experiments. The bitrate is controlled primarily through the entropy weight λ, while the remaining parameters are fixed across experiments.

3 Experimental set-up

We use two tools to compress the final weights: (i) a Range Coder and (ii) NNCodec. In this section, we describe the datasets and metrics used to carry out these experiments.

3.1 Datasets

We use a subset of DFaust [4], focusing on minimally clothed body geometry. The main experiments use the five female subjects, while additional results use all ten subjects. For validation and testing, the same two actions are selected for all characters—"running on spot" and "punching"—as they cover diverse upper- and lower-body motions. For training, nine actions per character are chosen. For simplicity, we focus on a subset of DFaust composed of the five female characters for the main experiments but also show some results obtained with the full DFaust dataset. Quantitative results are all computed on unseen poses and identities, more precisely on the two left-out actions ("running on spot" and "punching") regenerated for novel identities using SMPL parameters [7]. We generate new identities by averaging the β parameters of each character pair, making ten new identities. The results obtained with the new identities are then averaged. Qualitative results are generated using the AIST++ demo dance sequence [5], or the *ulr1d* sequence of the MPI PosePrior dataset [2], which contains challenging poses far from the training distribution.

3.2 Metrics

Two quantitative metrics were employed: Intersection over Union (IoU) and Chamfer distance.

- **Intersection over Union:** Following Fast-SNARF [5], the IoU of correctly predicted occupancy points is computed for samples taken close to the surface (IoU_{surf}). Among all tested metrics (normal consistency, Hausdorff distance, F-score, etc.), IoU_{surf} seemed to be the most discriminative for high-frequency details.

- **Chamfer distance:** The Chamfer distance is computed between the simulated and ground-truth meshes. Twenty thousand points are uniformly sampled on the surface of the simulated mesh. The distance of each sampled point to the closest face of the ground-truth mesh is calculated. This approach is consistent for topologically different

meshes. Chamfer distance provides a simple yet intuitive measure of low-frequency reconstruction errors.

3.3 Bitstream coding

To compress the generative models after regularization, two state-of-the-art coding methods are considered:

- *Range Coder:* a lossless entropy encoder that is nearly optimal, i.e., the number of bits per symbol after encoding is almost equal to the entropy. Entropy represents the theoretical lower bound on the number of bits per symbol. The achievable compression rate is constrained by the number of quantization levels used. In this work, ten bits were used as the maximum quantization level.
- *NNCodec* [3]: a lossy encoder designed specifically for neural network weights. It includes a built-in quantization parameter qp, which enables fine-grained control over the reconstruction–bitrate trade-off. NNCodec accepts both pre-quantized and unquantized weights as input, performing its own quantization internally.

3.4 Soft-to-Hard implementation details

We reimplemented Soft-to-Hard [1]. In the original schedule, σ starts at 0.4, increases exponentially by approximately a factor of 20, and finally switches to hard assignment. LHB did not converge with this schedule. Our best stable configuration instead increased σ from 5 to 100 to respect the times 20 factor.

4 Additional results

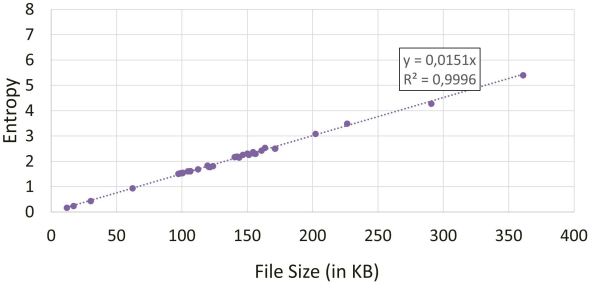


Figure 2: Estimated entropy versus file size after encoding with Range Coder across all our experiments, showing a clear linear correlation.

4.1 Entropy versus file size

Figure 2 plots the measured entropy against the range-coded file size in KB, with a fitted slope of 0.0151 bits per parameter per KB. This experimental proportionality is close to the theoretical scaling obtained by converting file size to bits and normalizing by the number of



Figure 3: Comparison of reconstructions produced by the different models at low bitrates. COAP [9], Fast-SNARF [5], LHB [8] are quantized to 7 bits to reduce the file size. The SMPL [7] mesh is shown on the right as reference. With $\lambda = 500$ and $q = 10$, our method achieves a similar file size with substantially higher reconstruction quality, illustrating the limitations of post-training quantization for these models.

parameters: $\frac{8000}{522600} \approx 0.0153$. This indicates that modern entropy coders can approach the theoretical lower bound in practice, making the entropy a useful proxy for the final file size.

4.2 Low bitrate

Figure 3 compares several neural avatar models (COAP [9], Fast-SNARF [5], and LHB [8]) at low bitrates. The SMPL mesh is shown on the right as a reference. All baseline methods exhibit strong distortions compared to the ground truth at similar bitrates. In contrast, LHB trained with our regularization achieves a reconstruction much closer to SMPL at a fraction of the size.

4.3 Large entropy weights

As mentioned in the main paper, training with an entropy regularization can lead to unstable behaviors. In our experiments, beyond a certain point, increasing λ results in higher entropy. Figure 5 presents results obtained with high λ. Up to $\lambda = 650$, both file size and IoU decrease as λ increases, reaching 114 KB. However, for $\lambda \geq 750$, the file size plateaus around 142 KB (equivalent to an entropy of 2.14). Figure 4 compares the final weight distribution of $\lambda = 650$ and $\lambda = 750$. Although both settings yield similar IoU, $\lambda = 750$ has higher entropy, consistent with its histogram showing more peaks. We hypothesize that large λ may over-regularize the training: the model first maps the weights to sub-optimal values and is trapped in a local minimum. Please see the supplementary video for the evolution of the weight distribution during training.

4.4 Regularization followed by PTQ

We compare two ways of controlling the bitrate: (i) varying λ across training runs while fixing $q = 10$, and (ii) reducing the quantization level q after training through PTQ (Post-Training Quantization). The objective is to identify the best combination of λ and q. In

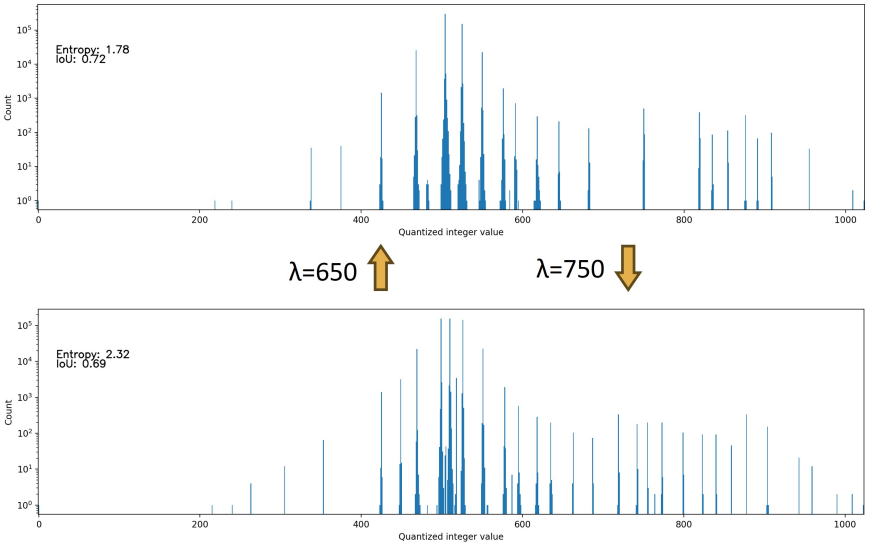


Figure 4: Histograms of the weight distribution at the end of the training for two different λ: 650 (top) and 750 (bottom). They both have similar IoU, although $\lambda = 750$ has a higher entropy.

practice, the model is trained with different values of λ, while the default quantization level is fixed at $q = 10$. Figure 6 reports results where each color corresponds to a distinct λ value. After training, each model is quantized to progressively lower levels. The best results were obtained at $q = 10$ with steeper decrease as λ increases. At higher λ, the model seems to focus on a few unique values obtained with $q = 10$. Changing the quantization level at inference (PTQ) can thus drastically worsen the reconstruction quality, supporting the hypothesis that fixing the quantization level to $q = 10$ and varying λ provides finer control over the reconstruction–rate trade-off.

References

[1] Eirikur Agustsson, Fabian Mentzer, Michael Tschannen, Lukas Cavigelli, Radu Timofte, Luca Benini, and Luc Van Gool. Soft-to-hard vector quantization for end-to-end learning compressible representations. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 1141–1151, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

[2] Ijaz Akhter and Michael J. Black. Pose-conditioned joint angle limits for 3D human pose reconstruction. In *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR) 2015*, June 2015.

[3] Daniel Becking, Paul Haase, Heiner Kirchhoffer, Karsten Müller, Wojciech Samek, and Detlev Marpe. NNCodec: An open source software implementation of the neural network coding ISO/IEC standard. In *ICML 2023 Workshop Neural Compression: From*

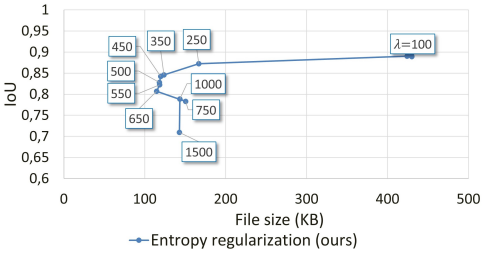


Figure 5: IoU versus file size for λ ranging from 100 to 1500.

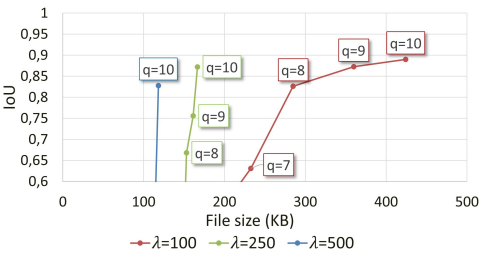


Figure 6: IoU versus file size in KB for different regularization weight λ. For each λ, the model is trained with $q = 10$ but tested with different quantization levels.

Information Theory to Applications, 2023. URL https://openreview.net/forum?id=5VgMDKUgX0.

[4] Federica Bogo, Javier Romero, Gerard Pons-Moll, and Michael J. Black. Dynamic FAUST: Registering human bodies in motion. In *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, July 2017.

[5] Xu Chen, Tianjian Jiang, Jie Song, Max Rietmann, Andreas Geiger, Michael J. Black, and Otmar Hilliges. Fast-SNARF: A fast deformer for articulated neural fields. *Pattern Analysis and Machine Intelligence (PAMI)*, 2023.

[6] Ruilong Li, Shan Yang, David A. Ross, and Angjoo Kanazawa. Learn to dance with aist++: Music conditioned 3d dance generation, 2021.

[7] Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J. Black. SMPL: A skinned multi-person linear model. *ACM Trans. Graphics (Proc. SIGGRAPH Asia)*, 34(6):248:1–248:16, October 2015.

[8] Guillaume Loranchet, Pierre Hellier, Adnane Boukhayma, João Regateiro, and Franck Multon. Neural implicit avatar conditioned on human pose, identity and gender. In *Proceedings of the 22nd ACM SIGGRAPH European Conference on Visual Media Production*, pages 1–11, 2025.

[9] Marko Mihajlovic, Shunsuke Saito, Aayush Bansal, Michael Zollhoefer, and Siyu Tang. COAP: Compositional articulated occupancy of people. In *Proceedings IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. IEEE, jun 2022. doi: 10.1109/cvpr52688.2022.01285.